UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE

Integrantes: Sebastian Oñate Ayña

Docente: Ing. Ruiz

NRC: 36460

FECHA: 07/10/2026

TEMA: 3

PROBLEMA 1.1

1. Analisis del problema

Entrada : num1, num2

Proceso : media ← (num1+num2)/2

Salida : media

Restriccion : No hay

2. Tabla de objetos

Objetos	Nombres	Valor	Tipo
Entrada	num1, num2	var	entero
Salida	media	var	real
Restriccion	-	-	-

3. Pseudocodigo

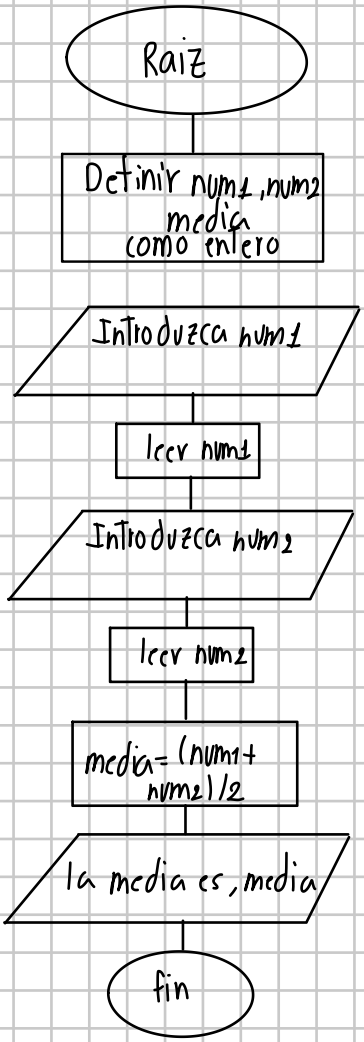
```
Algoritmo media
Definir num1, num2 como entero
Definir media como real
Escribir "Ingrese num1: ";
leer num1;
Escribir "Ingrese num2: ";
leer num2;
```

media <- (num1+num2)/2;

Escribir "la media es:", media;

fin algoritmo

4. Diagrama de flujo



5. Prueba de escritorio

num1	num2	media	salida
2	3	(2+3)/2	2.5
9	3	(9+3)/2	6
5	4	(5+4)/2	9.5

PROBLEMA 1.2

1. Analisis del problema

Entrada : N1

Proceso : Si n1 < 0 entonces Vabs es (-1)(n1*n1*n1) sino (n1*n1*n1) es Vabs

Salida : Vabs

Restriccion : No hay

2. Tabla de objetos

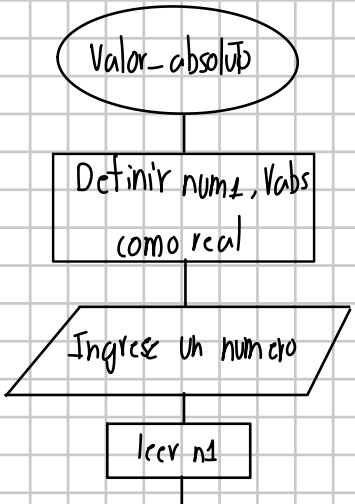
Objetos	Nombres	Valor	Tipo
Entrada	n1	var	real
Salida	Vabs	var	real
Restriccion	-	-	-

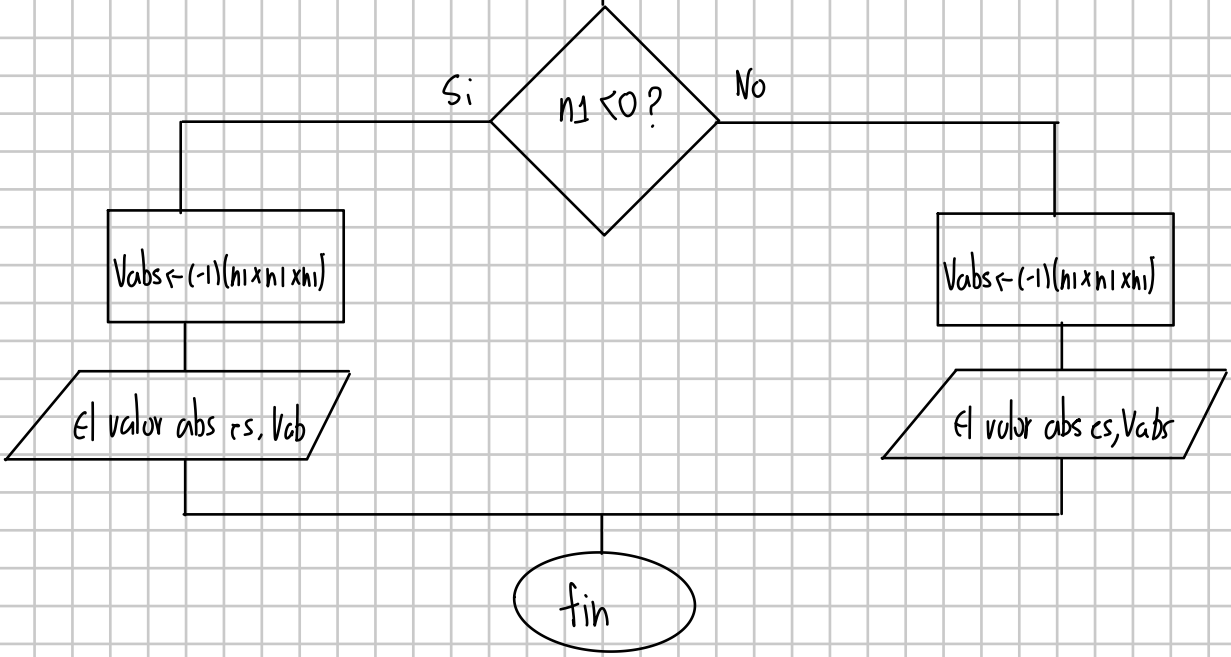
3. Pseudocodigo

```
Algoritmo Valor_absoluto

Definir n1, Vabs como real
Escribir "Ingrese un numero";
leer n1;
Si n1<0 entonces
  Vabs <- (-1)(n1*n1*n1)
  Sino
  Vabs<-(n1*n1*n1)
fin si
Escribir" El valor abs es: ", Vabs;
fin algoritmo
```

4. Diagrama de flujo





5. Prueba de escritorio

n1	n1<0?	Vabs	Salida
3	No	3x3x3	27
-2	Si	(-1)(-2x-2x-2)	8
0	No	0x0x0	0

PROBLEMA 1.3

1. Analisis del problema

Entrada : n1, n2

Proceso : Si n2 % n1 es 0 entonces es divisible sino no

Salida : -

Restriccion : n1 ≠ 0

2. Tabla de objetos

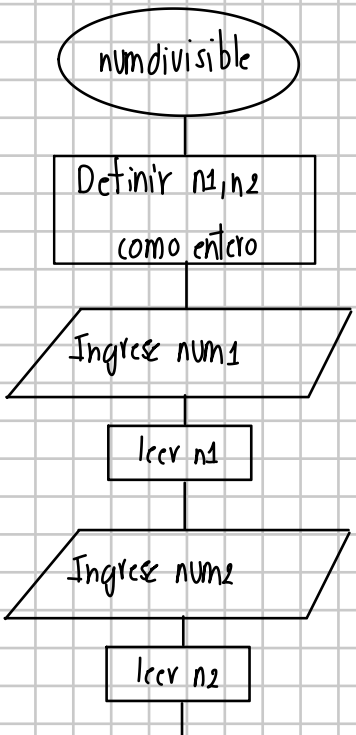
Objetos	Nombres	Valor	Tipo
Entrada	n1, n2	var	entero
Salida	-	-	-
Restriccion	n1 ≠ 0	cond	bool

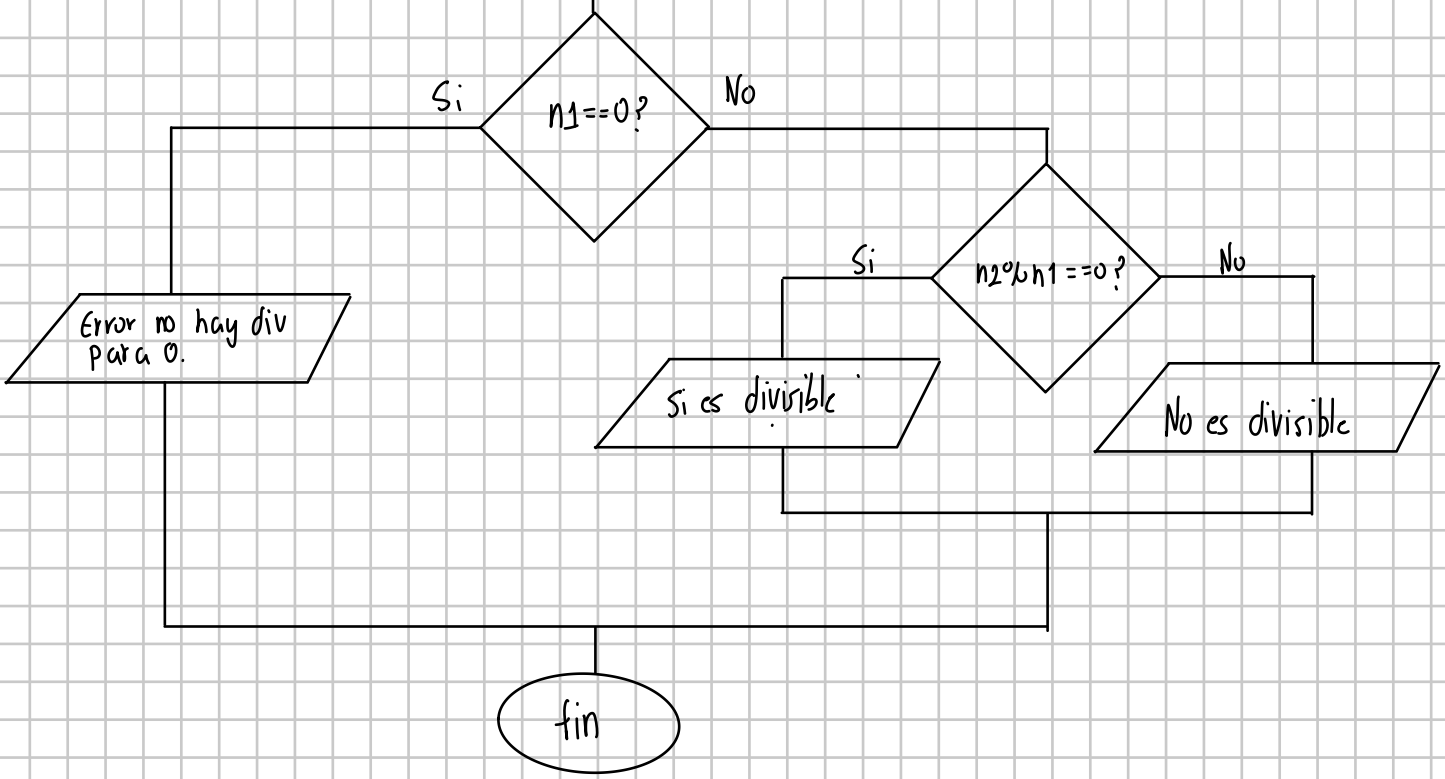
3. Pseudocodigo

```
Algoritmo numdivisible
  Definir n1,n2 como entero;
  Escribir "Ingrese num1 : ";
  leer n1;
  Escribir "Ingrese num2 : ";
  leer n2;
  Si n1==0 entonces
    Escribir "Error";

  Si no
    Si n2%n1==0 entonces
      Escribir "Si es divisible";
      Si no
      Escribir "No es divisible";
    fin si
  fin si
fin algoritmo
```

4. Diagrama de flujo





5. Prueba de escritorio

n1	n2	n1==0	n2%n1==0	Salida
2	3	F	F	No es divisible
0	1	V	-	Error
2	4	V	V	Si es divisible

PROBLEMA 1.4

1. Analisis del problema

Entrada : n1

Proceso : Si n1 esta entre n1%10 ^ n2≤10 entonces ∈ al intervalo

Salida : —

Restriccion : No hay

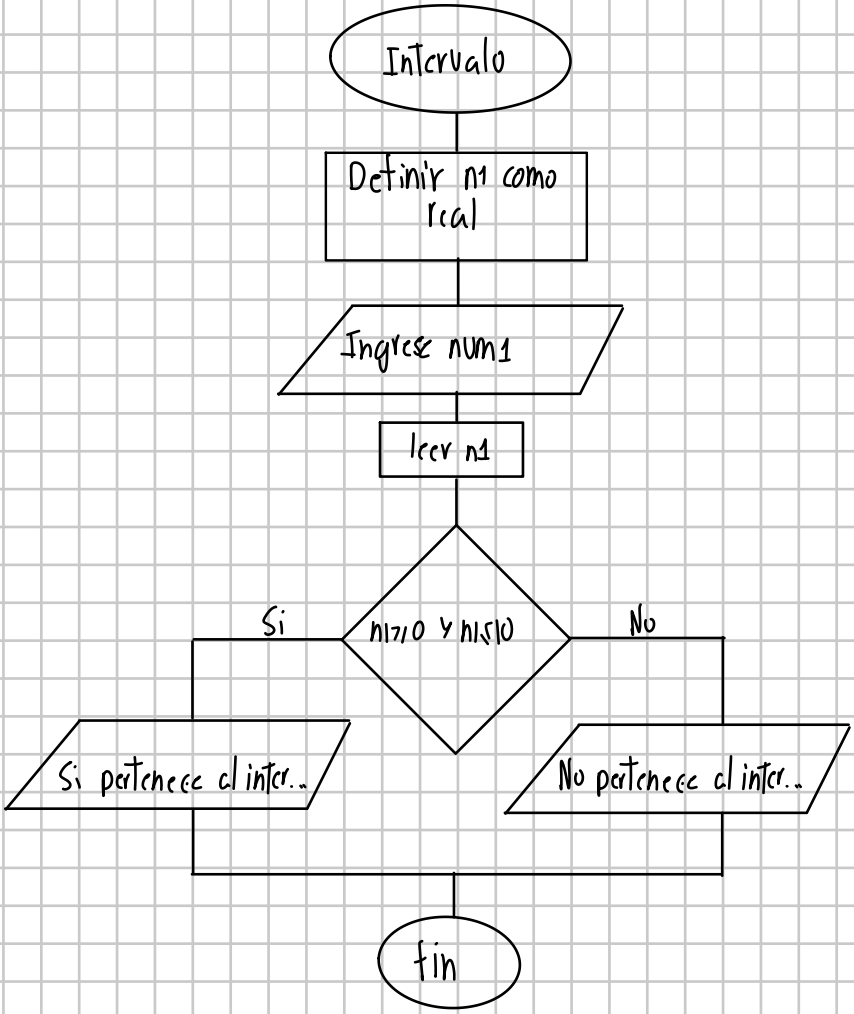
2. Tabla de objetos

Objetos	Nombres	Valor	Tipo
Entrada	n1	Var	entero
Salida	-	-	-
Restriccion	-	-	-

3. Pseudocodigo

```
Algoritmo intervalo
Definir n1,n2 como real;
Escribir "Ingrese num1: ";
leer n1;
Si n1>=10 y n1<=0 entonces
  Escribir "Si Pertenece al intervalo";
  Si no
  Escribir "No pertenece al intervalo";
fin si
fin algoritmo
```

4) Diagrama de flujo



5. Prueba de escritorio

n1	n1 ≥ 0 ^ n1 ≤ 10	Salida
3	V ^ V = V	Pertenece al intervalo
-2	F ^ V = V	No pertenece al intervalo
0.1	V ^ V = V	Pertenece al intervalo

PROBLEMA 1.5

1. Analisis del problema

Entrada : c es la cantidad de segundos

Proceso : Tomamos c ^ dividimos para 86400, tomamos el resto ^ hacemos lo mismo para 3600, luego para 60

Salida : dias, horas, min, c

Restriccion : c > 0

2. Tabla de objetos

Objetos	Nombres	Valor	Tipo
Entrada	c	var	entero
Salida	dias, horas, min, c	var	entero
Restriccion	c≤0	cond	bool

3. Pseudocodigo

Algoritmo Conversión

Definir c, dias, horas, min como entero;

Escribir "Ingrese la cantidad de seg:";

leer c;

Si c≤0 entonces

Escribir "Error no hay Tiempo negativo!";

Si no

dias ← Trunc(c / 84.600);

c ← c % 84600;

horas ← Trunc (c / 3600);

c ← c % 3600;

min ← Trunc (c / 60);

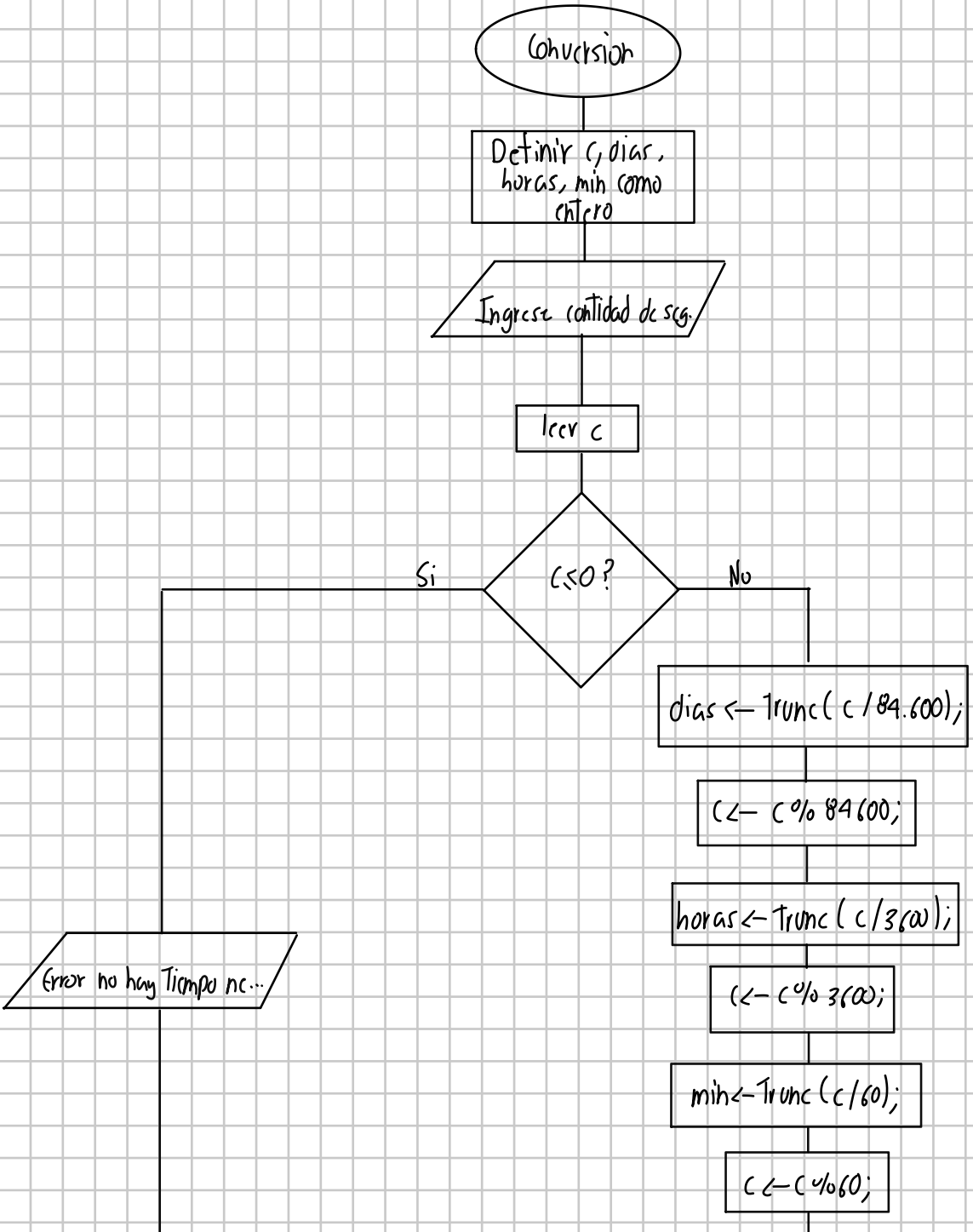
c ← c % 60;

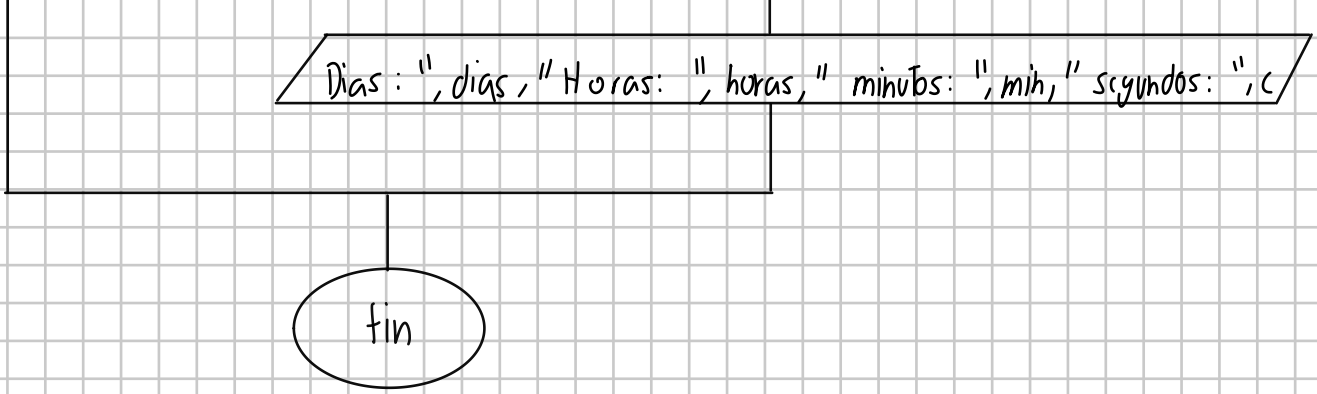
Escribir " Dias : ", dias, " Horas: ", horas, " minutos: ", min, " segundos: ", c;

fin algoritmo

4. Dia rama de H

g yo





5. Prueba de escritorio

c	dias	horas	min	seg	salida
84664	84664 | 86400 64 | 1	64 | 3600 64 | 0	64 | 60 4 | 1	c<-4	dias:1 horas:0 min:1 seg:4
342	342 | 86400 342 | 0	342 | 3600 342 | 0	342 | 60 42 | 5	c<-42	dias:0 horas:0 min:5 seg:42
-3	-	-	-	-	Error

PROBLEMA 1.6

1. Analisis del problema

Entrada : alturas

Proceso : Asignamos el max y min con alt luego comparamos si esc/ max y min mientras avanzita la suma y contador para sacar la media.

Salida : media, max, min

Restriccion : alt>=0

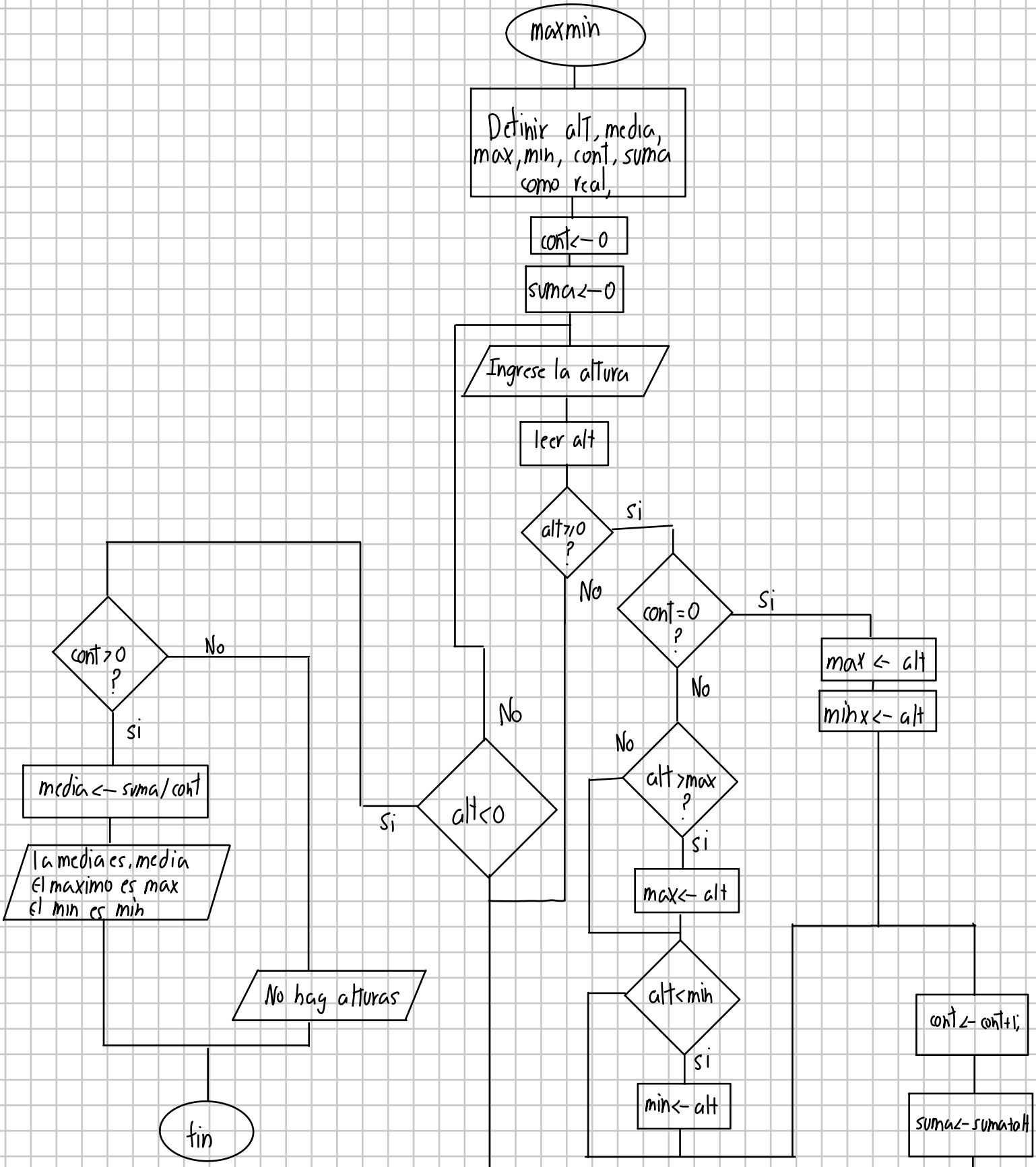
2. Tabla de objetos

Objetos	Nombres	Valor	Tipo
Entrada	alt	var	real
Salida	media, max, min	var	real
Restriccion	alt >=0	cond	bool

3. Pseudocodigo

```
Algoritmo maxmin
Definir alt, media, max, min, cont, suma como real,
cont<-0;
suma<-0;
Repetir
Escribir "Ingrese la altura";
leer alt;
Si alt >0 entonces
   Si cont==0 entonces
      max <- alt;
      minx<- alt;
   Si no
      Si alt >max entonces
         max <- alt;
      fin si
      Si alt< min entonces
         min<-alt;
   fin si
   suma<-suma+alt;
   cont<- cont+1;
fin si
hasta que alt<0;
Si cont>0 entonces
   media<-suma/cont;
Escribir" la media es: ", media," el max es: ", max," el min es: ", min;
Si no
Escribir "No hay alturas";
fin si
fin algoritmo
```

4. Diagrama de flujo



5. Prueba de escritorio

alt	cont	suma	max	min
-	0	0	-	-
10	1	10	10	10
20	2	30	20	10
-1	2	30	20	10
				fin

PROBLEMA 7.

1. Analisis del problema

Entrada : N es el numero que vamos a sacar raiz

Proceso : i inicia en 1 e itera hasta que i*i ≤ N

Salida : Raiz que es el valor de i-1

Restriccion : N >= 0

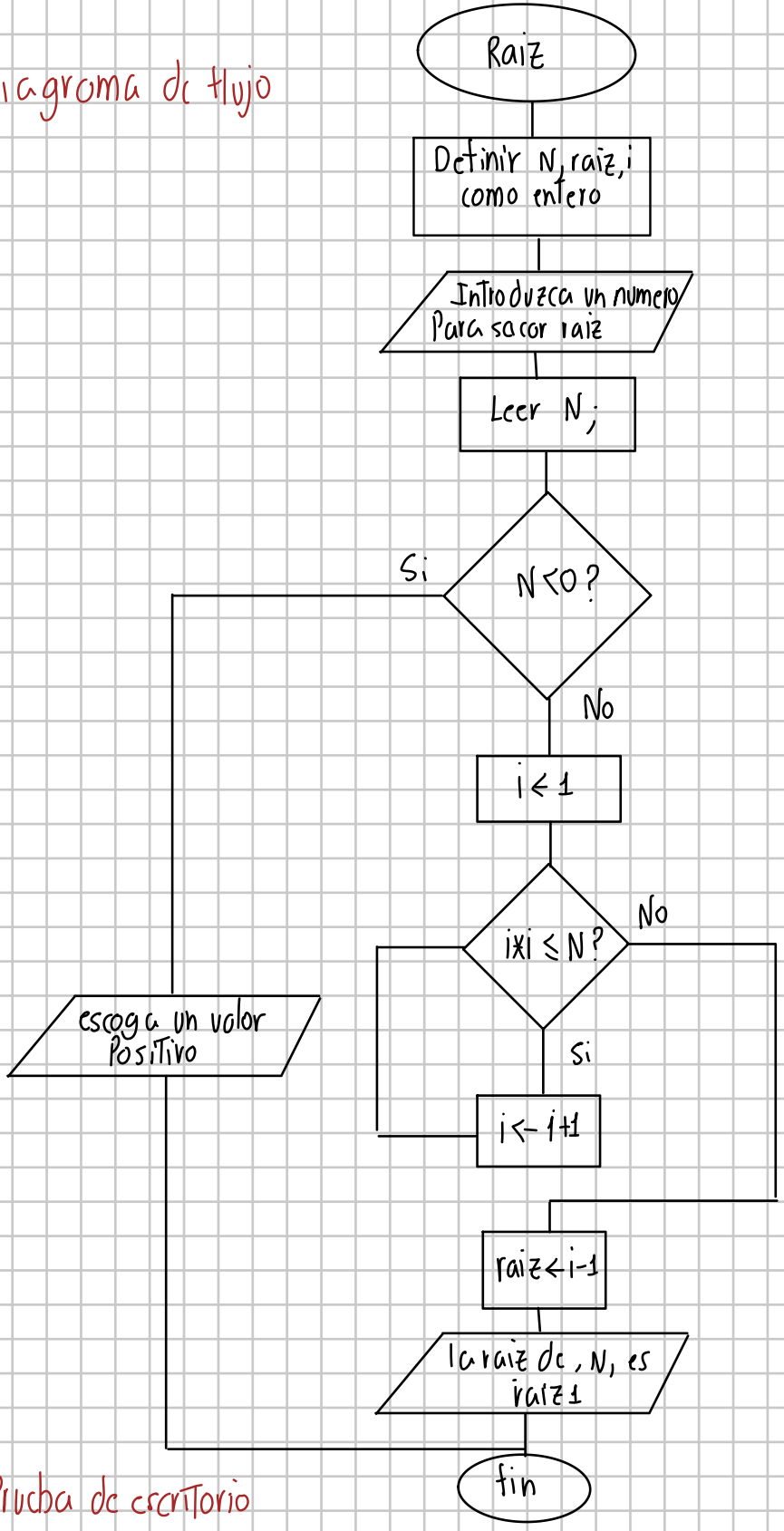
2. Tabla de objetos

Objetos	Nombres	Valor	Tipo
Entrada	N	var	entero
Salida	raiz1	var	entero
Restriccion	N >= 0	cond.	V/F

3. Pseudocodigo

```
Algoritmo Raiz.
  Definir N, raiz1, i como entero;
  Escribir "Introduzca el numero para sacar raiz: ";
  leer N;
  si N < 0 entonces
    Escribir "escoga un valor positivo";
    Sino
    i <- 1;
   mientras i*i ≤ N hacer
        i <- i+1;
    fin mientras
    Raiz1 <- i-1;
   Escribir "la raiz de ", N, " es: ", raiz1;
fin si
 fin algoritmo
```

4. Diagroma de flujo



5. Prueba de escritorio

N	N≤0	i<-1	i*i≤N	Raiz ← i-1;
3	F	1 2	1 ≤ 3 4 ≤ 3 ← falso	2-1 = 1
4	F	1 2 3	1 ≤ 4 4 ≤ 4 9 ≤ 4 ← falso	3-1 = 2
0	F	1	1 ≤ 0 ← falso	1-1 = 0

PROBLEMA 1.8

1. Analisis del problema

Entrada : num es la var que almacenara el valor ingresado

Proceso : num tiene que ser dividido entre 2 hasta num ^ buscar cuando el modulo = 0 si pasa de 2 entonces no es primo si es 1 es primo

Salida : Si es primo o no el num

Restriccion : que num ≤ 1

2. Tabla de objetos

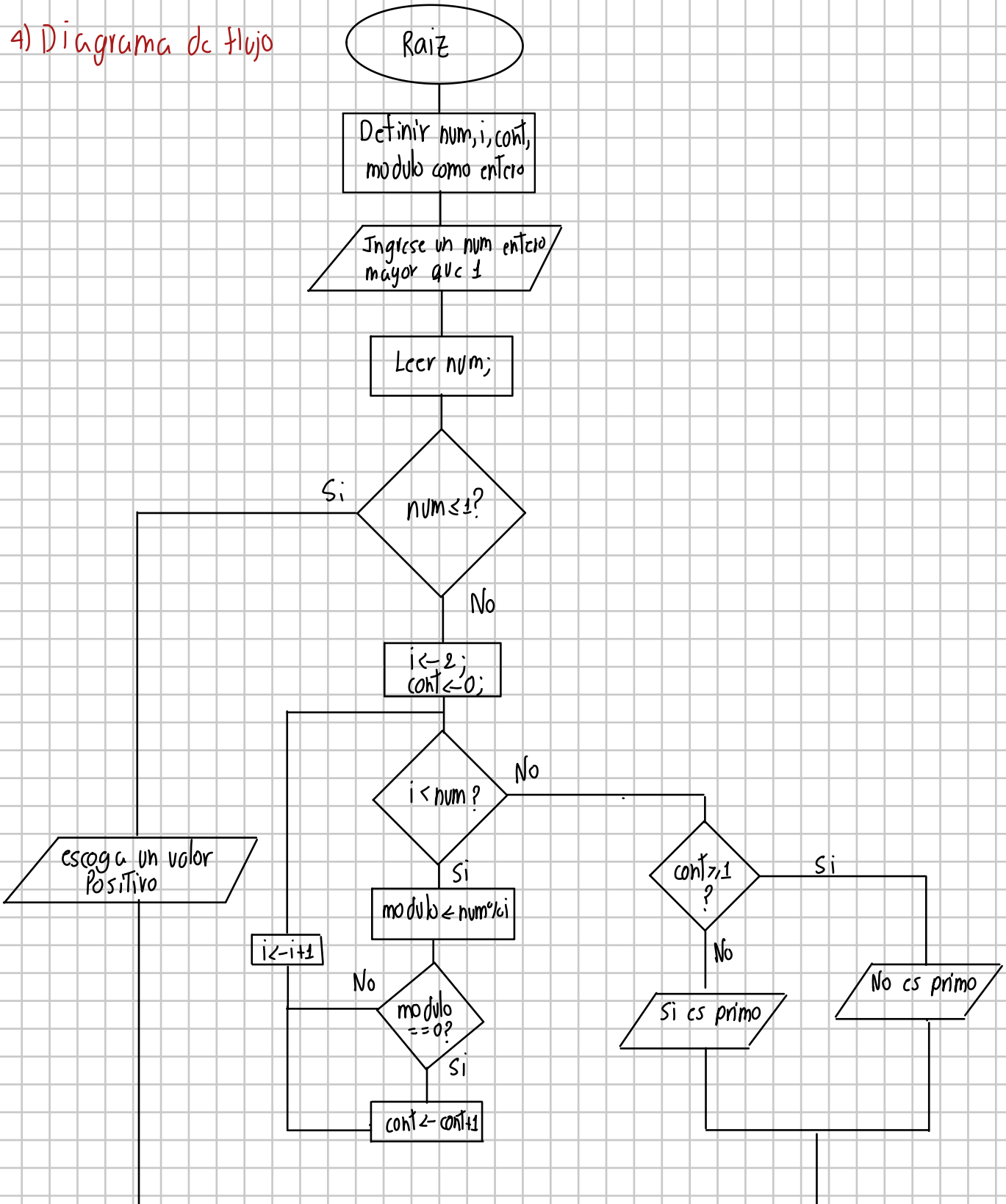
Objetos	Nombres	Valor	Tipo
Entrada	num	var	entero
Salida	—	—	—
Restriccion	num ≤ 1	cond.	V/F

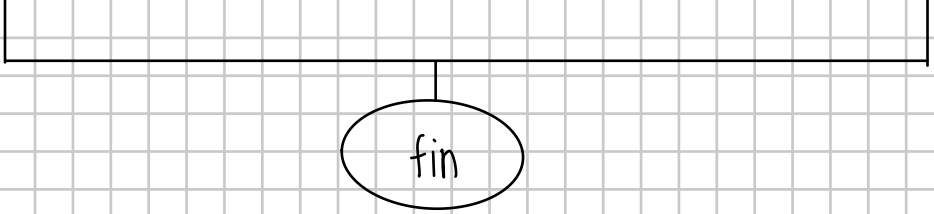
3. Pseudocodigo

```
Algoritmo Primo
  Definir num, i, cont, modulo como entero; // 7
  Escribir "Ingrese un numero entero mayor que 1";
  leer num;
  si num ≤ 1 entonces
    Escribir "Escoga un valor correcto";
    Sino
      i<- 2;
      cont<-0;
    mientrasi < num hacer    // 2 ≤ 2
     modulo ← num % i; // modulo <-0
     si modulo ==0 entonces
        cont<- cont+1;   // cont<-1
      fin si
          i<- i+1;
  fin mientras
```

Si cont > 1 entonces
Escribir "No es primo";
Sino
Escribir "Es primo";
fin si
fin si
fin algoritmo

4) Diagrama de flujo





5) Prueba de escritorio

num	num ≤ 1	i < num	modulo ← num%1	cont	Salida
2	F	2 < 2	—	No	Si es primo
0	V				fin
3	F	2 < 3 3 < 3	3%2 == 1 F	No	Si es primo
4	F	2 < 4 3 < 4 4 < 4	4%02 == 0 V 4%03 == 1 F	Si	No es primo

PROBLEMA 1.9

1. Analisis del problema

Entrada : num1, num2 son los rangos para buscar los primos

Proceso : i ← num1 ^ luego iteramos aplicando % ^ verificando si es primo o no si si es imprimimos el numero sino aumenta i+1 hasta llegar a num2

Salida : Son los numeros primos

Restriccion : que num1 ^ num2 > 1

2. Tabla de objetos

Objetos	Nombres	Valor	Tipo
Entrada	num1, num2	var	entero
Salida			
Restriccion	num1 ^ num2 > 1	cond.	bool

3. Pseudocodigo

Algoritmo Busqueda-primos

Definir num1, num2, cont, i, modulo, aux como enteros;

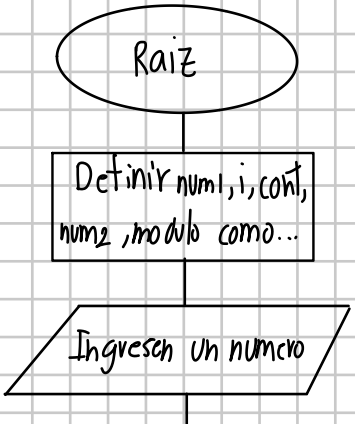
Escribir "Ingrese num1 : ";

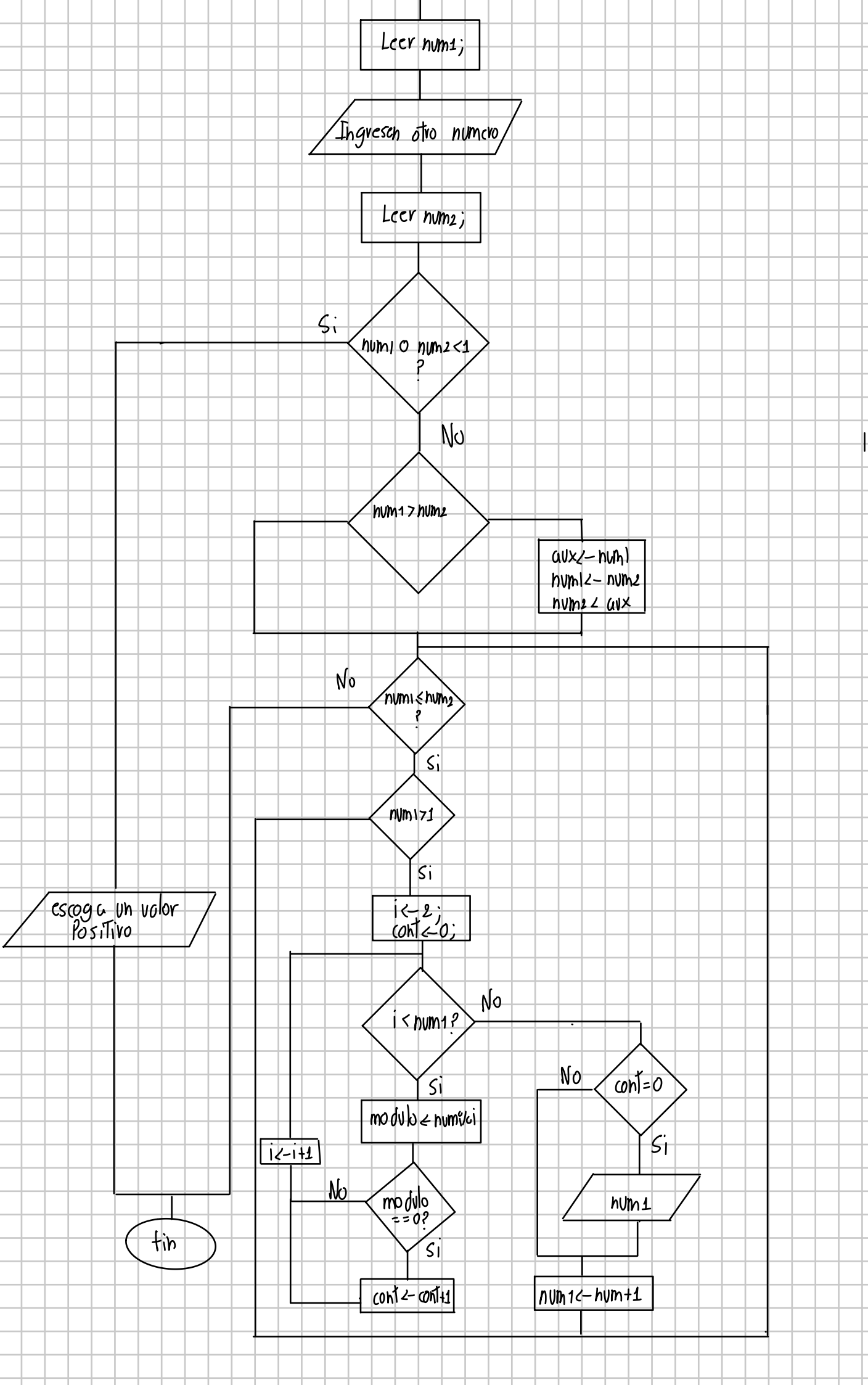
leer num1;

Escribir "Ingrese num2 : ";

```
leer num2;
si num1 o num2 < 1 entonces
  Escribir " Escoga numeros positivos";
  si no
    Si num1>num2 entonces
      aux<— num1
      num1<— num2
      num2<— aux
    fin si
  mientras num1 ≤ num2 hacer
    Si num1 > 1 entonces
        i<—2;
        cont<—0;
      mientras i < num1 hacer
        modulo<— num1 MOD i;
        si modulo ==0 entonces
          cont<— cont+1;
        fin si
        i<— i+1;
      fin mientras
      Si cont==0 entonces
        Escribir " ", num1;
      fin si
    fin si
    num1<—num1+1;
  fin mientras
fin si
fin algoritmo
```

4. Diagrama de flujo




Leer num1;
Ingresar otro numero
Leer num2;
Si
num1 o num2 <1 ?
No
num1 > num2
aux <- num1
num1 <- num2
num2 <- aux
No
num1 ≤ num2 ?
Si
num1 > 1
Si
escoga un valor positivo
i <- 2;
cont <- 0;
i < num1 ?
No
Si
modulo <- num%i
i <- i+1
No
modulo == 0?
Si
cont <- cont+1
No
cont = 0
Si
num1
num1 <- num+1
fin

5. Prueba de escritorio

num1	num2	num1 o num2 < 1	num1 > num2	num1 <= num2	num1 > 1	i	cont	salida
2	3	F	F	V	V	i < num1 2 < 2		2
				V	V	2 < 3 2 | 3 [2] | 0 3 < 3		3
-7	2	V	-	-	-	-	-	-

PROBLEMA 1.10

1. Analisis del problema

Entrada : num1 para sacar factorial

Proceso : iteramos i hasta que llegue a num1 multiplicando los valores de i

Salida : fact

Restriccion : fact < 0

2. Tabla de objetos

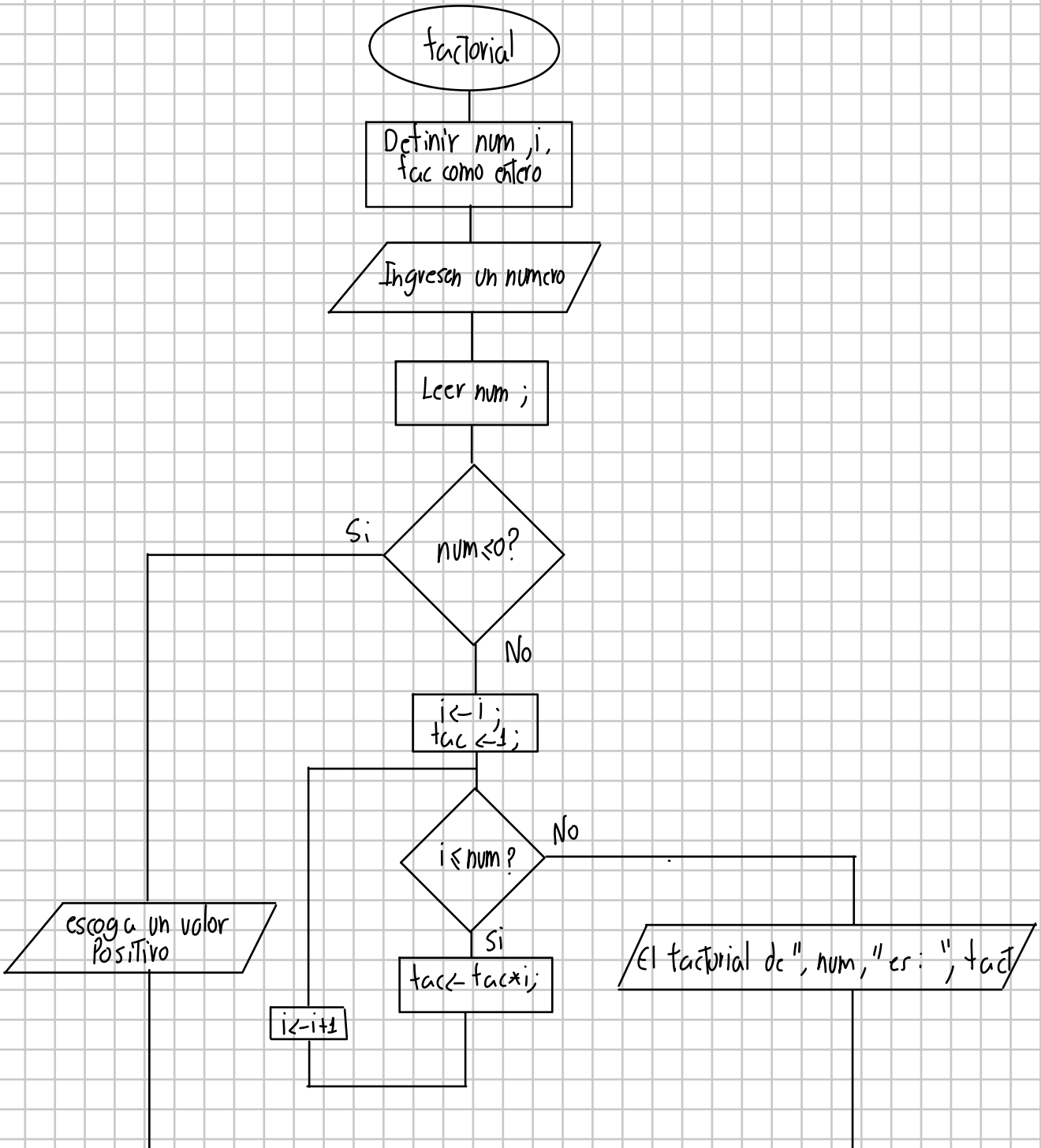
Objetos	Nombres	Valor	Tipo
Entrada	num1	var	entero
Salida	fact	var	entero
Restriccion	num >= 0	cond	bool

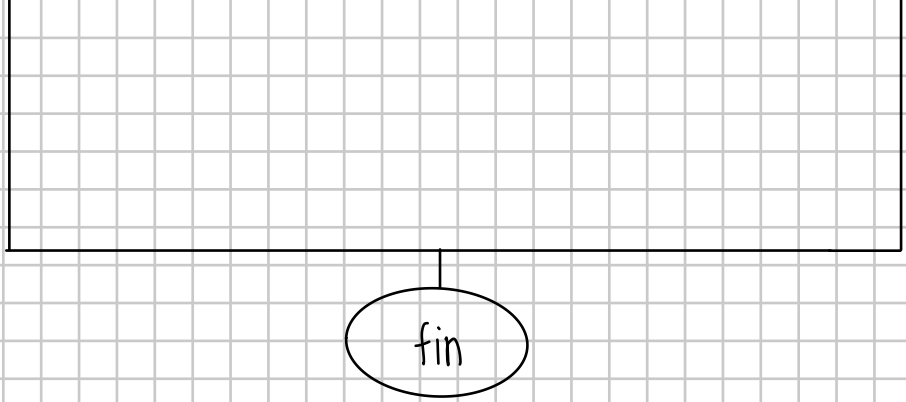
3. Pseudocodigo

Algoritmo factorial

```
Definir num,i como entero;
Definir fac como entero;
Escribir "Ingrese num : ";
leer num ;
Si num <0 entonces
Escribir " Escoga numeros positivos";
```

```
Si no
i<- 1;
fac<-1;
  mientras i ≤ num hacer
    fac<- fac * i;
    i<-i+1;
  fin mientras
  Escribir " El factorial de ", num, " es: ", fac";
fin si
fin algoritmo
```





5. Prueba de escritorio

num	num<0?	i≤num?	fac	salida
4	F	1≤4	1x1=1	
		2≤4	2x1=2	
		3≤4	2x3=6	
		4≤4	6x4=24	24
		5≤4 F	-	
-1	V	-	-	-
2	F	1≤2	1x1=1	
		2≤2	2x1=2	2
		3≤2	-	